NSDA Assessment — SQL Practical Question Bank (30 Questions)

Topics: CRUD Operations | Joins | Views | Stored Procedures | Functions

These are hands-on tasks — write and, where possible, test the actual T-SQL code.

Reference Schema

Use this schema as the basis for all practical questions below (create it first in SSMS / Azure Data Studio):

```
CREATE TABLE Clients (
    ClientId    INT IDENTITY(1,1) PRIMARY KEY,
    Name        NVARCHAR(100) NOT NULL,
    Email       NVARCHAR(100),
    Phone       NVARCHAR(20),
    IsActive    BIT DEFAULT 1,
    JoinDate    DATE DEFAULT GETDATE()
);

CREATE TABLE Employees (
    EmployeeId  INT IDENTITY(1,1) PRIMARY KEY,
    Name        NVARCHAR(100) NOT NULL,
    Department  NVARCHAR(50),
    Salary      DECIMAL(10,2),
    ManagerId   INT NULL REFERENCES Employees(EmployeeId)
);

CREATE TABLE Invoices (
    InvoiceId   INT IDENTITY(1,1) PRIMARY KEY,
    ClientId    INT NOT NULL REFERENCES Clients(ClientId),
    Amount      DECIMAL(10,2) NOT NULL,
    InvoiceDate DATE DEFAULT GETDATE(),
    Status      NVARCHAR(20) DEFAULT 'Unpaid'
);
```

Section A: CRUD Practical Tasks (Q1–Q6)

1. Insert 5 sample rows into `Clients` with realistic names, emails, and phone numbers.
2. Insert 3 invoices for one of the clients you just created, with different `Amount` and `Status` values.
3. Write a query to update the `Status` of all invoices older than 30 days from today to `'Overdue'`.
4. Write a query to soft-delete a client (set `IsActive = 0`) instead of physically deleting the row. Explain why soft-delete is often preferred in production systems.
5. Write a query to permanently delete all invoices with `Status = 'Cancelled'`.
6. Write a single query that inserts a new client **and** returns the newly generated `ClientId` using `OUTPUT` (or `SCOPE_IDENTITY()`).

Section B: Join Practical Tasks (Q7–Q13)

7. Write a query using `INNER JOIN` to list every invoice with the corresponding client's `Name` and `Email`.
8. Write a query using `LEFT JOIN` to list **all** clients and their invoice count, including clients with zero invoices (hint: use `COUNT` + `GROUP BY`).
9. Write a query to find all clients who have **never** been invoiced (no matching row in `Invoices`), using either `LEFT JOIN ... IS NULL` or `NOT EXISTS`.
10. Using the `Employees` table, write a `SELF JOIN` query that lists each employee alongside their manager's name.
11. Write a query that joins `Clients` and `Invoices` to calculate the **total amount invoiced per client**, sorted from highest to lowest.

12. Write a query using `JOIN` and `WHERE` to find all clients whose total unpaid invoice amount exceeds 10,000.
13. Modify Q7 into a `FULL OUTER JOIN` and explain, with the actual result set, how it differs from the `INNER JOIN` version.

Section C: View Practical Tasks (Q14–Q19)

14. Create a View named `vw_ActiveClients` that returns only clients where `IsActive = 1`.
15. Create a View named `vw_ClientInvoiceSummary` that shows `ClientId`, `Name`, total invoiced amount, and total unpaid amount per client (using `JOIN` + aggregate functions).
16. Write a query that selects from `vw_ClientInvoiceSummary` to find the top 3 clients by total invoiced amount.
17. Attempt to `UPDATE` a client's `Email` through `vw_ActiveClients`. Does it succeed? Explain why or why not.
18. Alter `vw_ActiveClients` (using `ALTER VIEW`) to also include the `Phone` column.
19. Write the SQL to drop the `vw_ClientInvoiceSummary` view, then recreate it using `CREATE OR ALTER VIEW`.

Section D: Stored Procedure Practical Tasks (Q20–Q25)

20. Create a stored procedure `sp_GetClientById` that takes `@ClientId INT` as a parameter and returns that client's full record.
21. Create a stored procedure `sp_AddClient` that accepts `@Name`, `@Email`, `@Phone` as input parameters, inserts a new client, and returns the new `ClientId` via an `OUTPUT` parameter.
22. Create a stored procedure `sp_UpdateInvoiceStatus` that accepts `@InvoiceId` and `@NewStatus`, updates the invoice, and returns the number of rows affected.
23. Create a stored procedure `sp_GetOverdueInvoices` that returns all invoices with `Status = 'Overdue'` joined with client name and email.
24. Wrap the insert logic in `sp_AddClient` with `TRY...CATCH` and `BEGIN TRANSACTION` / `COMMIT` / `ROLLBACK` so that failures don't leave partial data.
25. Write the C# (ADO.NET or EF Core) code snippet you would use to call `sp_GetClientById` from a .NET 8 application and map the result to a `Client` model.

Section E: Function Practical Tasks (Q26–Q30)

26. Create a Scalar Function `fn_GetClientAge` — wait, since `Clients` has no birth date, instead create `fn_GetClientTenureDays` that takes `@JoinDate DATE` and returns the number of days since that client joined (using `DATEDIFF`).
27. Create a Scalar Function `fn_GetOutstandingBalance` that takes `@ClientId INT` and returns the total unpaid invoice amount for that client.
28. Create an Inline Table-Valued Function `fn_GetInvoicesByClient` that takes `@ClientId INT` and returns all invoices for that client.
29. Write a query that uses `fn_GetInvoicesByClient` with `CROSS APPLY` to show, for every client, their most recent invoice.
30. Compare calling `fn_GetOutstandingBalance` inside a `SELECT` column list (per-row) versus calling `sp_GetOverdueInvoices` as a full procedure — write both queries and explain which approach is more efficient for a report showing balances for 10,000 clients.

Practical Exam Tips

- Always run each script in SSMS/Azure Data Studio yourself before the assessment — NSDA practical rounds are graded on **working code**, not just syntax.
- Practice writing these **without IntelliSense** at least once, since some assessment environments have limited autocomplete.
- Be ready to explain your `JOIN` choice out loud (INNER vs LEFT) — examiners often ask "why not the other type?" as a follow-up.
- For Stored Procedures/Functions, know the exact keyword differences: `CREATE PROCEDURE` vs `CREATE FUNCTION`, `RETURNS TABLE` vs `RETURNS INT`, and that functions cannot use `INSERT` / `UPDATE` / `DELETE` against persistent tables.